

The Architecture of Materiality: A Comprehensive Analysis of Tactile Surface Morphisms and Multi-Sensory UI Implementations

The trajectory of user interface design has transitioned from the functional minimalism of the mid-2010s toward a sophisticated revival of materiality, depth, and tactile immersion. This shift is characterized by the emergence of "morphisms"—design movements that leverage modern browser capabilities to simulate the physical properties of light, volume, and texture. While the early digital era relied on skeuomorphism to bridge the gap between physical tools and virtual interfaces, the contemporary landscape utilizes claymorphism, glassmorphism, and various tactile surface treatments to create "sensory-rich" environments.¹ This report provides an exhaustive technical and theoretical analysis of these implementations, exploring the underlying code stacks, build architectures, and the psychological impact of tactile feedback in modern software.

The Philosophical Evolution of Digital Surfaces

To analyze the current state of tactile UI, one must first recognize the cyclical nature of design trends. Skeuomorphism, which dominated the early smartphone era, utilized realistic textures like wood, leather, and metal to provide immediate affordance and familiarity for users new to touch interfaces.¹ This was succeeded by flat design, which prioritized utility and vector-based scalability but often sacrificed the visual cues necessary for establishing hierarchy and interactivity.⁴

The limitations of flat design led to the brief rise of neumorphism in 2020, a style that attempted to make elements appear as if they were extruded from the background.¹ However, neumorphism was widely criticized for significant accessibility failures, particularly regarding low contrast and readability for visually impaired users.² Claymorphism and its successors have addressed these deficiencies by reintroducing high-contrast borders, vibrant color palettes, and distinct layering while maintaining the soft, approachable aesthetic that users find engaging.¹

Claymorphism: Technical Foundations and CSS Implementation

Claymorphism is defined by its "inflated" 3D appearance, mimicking the look of modeling clay. Unlike previous trends, it does not attempt to merge elements with the background; instead, claymorphic elements appear to float over the UI with their own internal volume.³ This effect is achieved primarily through the strategic layering of multiple box shadows and the use of "squircles"—shapes that utilize continuous curvature rather than standard rounded corners.⁸

Core CSS Architecture for Claymorphic Elements

The implementation of a claymorphic card requires a combination of box-shadow properties that include both inner (inset) and outer shadows. The outer shadow provides depth and separation from the base layer, while the inner shadows simulate the curvature and internal light reflection of a soft material.¹

CSS Component	Functional Description	Typical Attribute Value
Primary Background	Pastel or vibrant base color	background: #D6A2FF ³
Outer Shadow	Establishes depth and float	0 20px 40px rgba(0,0,0,0.2) ⁸
Inset Shadow (Bottom)	Simulates volume and occlusion	inset -8px -16px 32px rgba(0,0,0,0.1) ⁸
Inset Shadow (Top)	Simulates light source reflection	inset 8px 16px 32px rgba(255,255,255,0.5) ¹
Border Radius	Continuous curvature (Squircle)	border-radius: 30px to 50px ⁸

The "squishiness" associated with claymorphism is not just a visual property but a psychological one. Large rounded corners and soft lighting suggest a material that is pliable and "friendly," which designers use to reduce user anxiety in complex workflows.¹ From a technical standpoint, creating the "perfect" clay shadow involves balancing the X and Y offsets to match a consistent global light source—typically from the top or top-left.¹

Squirclemorphism and Geometric Smoothing

A critical advancement in claymorphism is the transition from standard rounded rectangles to

squircles. While a standard rounded corner has a static radius that transitions abruptly into a straight line, a squircle follows a mathematical curve that ensures the rate of change in curvature is constant.⁸ In Figma, this is managed through "corner smoothing" settings, which adjust the Bezier handles of the radius to eliminate the visual "break" at the tangent point.⁸ In web development, this is often approximated using the clip-path property or by applying a radius that is significantly larger than the padding, creating a bloated, organic silhouette.⁷

Build Stacks for Tactile Interactivity

A static "morphism" provides visual appeal, but true tactile immersion requires the interface to respond to user input with physical realism. This necessitates an animation stack that incorporates spring physics and gesture-based interaction.

Spring Physics and Kinetic Feedback

Standard CSS transitions operate on a linear or cubic-bezier timeline, which can feel robotic. Tactile UIs utilize spring physics to simulate mass, tension, and friction.¹⁰ The behavior of a "squishy" button in a claymorphic system can be modeled using the damped harmonic oscillator formula:

$$m \frac{d^2x}{dt^2} + c \frac{dx}{dt} + kx = 0$$

In this equation, m represents the mass of the UI element, c is the damping (resistance), and k is the stiffness of the "spring".¹³ Libraries such as Framer Motion and React Spring allow developers to define these parameters directly, creating interactions where elements overshoot their target slightly and settle into place with natural momentum.¹⁰

Animation Library	Key Strength	Ideal Use Case
Framer Motion	Declarative API, gesture support	React-based high-end tactile UIs ¹⁰
React Spring	High performance, complex physics	Data-heavy interactive dashboards ¹²

Three.js / R3F	True 3D rendering and shaders	Silicon or Gel-like fluid surfaces ¹⁵
GSAP	Robust timeline and physics plugin	Complex chained UI sequences ¹³

Framer Motion, in particular, has become the industry standard for tactile UIs due to its `whileHover` and `whileTap` props, which provide immediate visual and kinetic feedback.¹⁰ This library also supports `AnimatePresence`, enabling "graceful exits" where elements animate out of the DOM before unmounting, preventing the jarring "disappearing" effect common in traditional web apps.¹⁰

React Three Fiber and 3D Morphisms

For more advanced tactile effects, such as "siliconmorphism" or "gelmorphism," developers are increasingly turning to React Three Fiber (R3F)—a React renderer for Three.js.¹⁵ This allows UI elements to be rendered as actual 3D meshes with custom shaders that simulate refraction and subsurface scattering.¹⁵ A "gel" button rendered in R3F can realistically distort the background behind it, behaving like a physical droplet of liquid or silicone.¹⁸

SVG Texture Engineering: Fabricmorphism and Papermorphism

While claymorphism and glassmorphism focus on volume and transparency, fabricmorphism and papermorphism introduce the dimension of physical texture. These styles aim to replicate the "grain" of physical materials like linen, cardstock, or quilled paper.¹⁹

The Role of SVG Filter Primitives

The technical core of digital texture generation is the `<feTurbulence>` filter. This primitive generates Perlin noise, which is used to create organic patterns that simulate the irregularities of natural materials.¹⁹ By combining turbulence with `<feDiffuseLighting>`, developers can create a "bump map" effect where the UI surface appears to have physical depth.¹⁹

Attributes of `<feTurbulence>` for Texture Control:

- **baseFrequency:** Determines the grain size. Low values (0.01) create large ripples (fabric); high values (0.5) create fine grit (paper).¹⁹
- **numOctaves:** Defines the detail level. Higher values result in more complex, realistic textures.¹⁹

- **type:** fractalNoise is generally preferred for smooth, cloudy textures like paper, while turbulence creates more linear, liquid-like patterns.¹⁹

Papercutmorphism and Quillingmorphism

"Papercutmorphism" mimics the effect of stacked paper layers, often seen in high-end editorial web design. This relies on the `<feDropShadow>` filter with high opacity and small blur to simulate the sharp edges of cardstock.⁴ "Quillingmorphism" is a decorative subset that replicates the art of quilling—rolled strips of paper. Implementing this style requires complex SVG pathing and multi-stop gradients to simulate the "edge" and "shadow" of a paper coil.²¹ These styles often leverage the CSS `z-index` property combined with `transform: translateZ()` to create a real sense of spatial stacking.²³

Glassmorphism, Cloudmorphism, and Atmospheric Depth

Glassmorphism, which rose to prominence as the defining aesthetic of 2021, remains a foundational style for modern dashboards and overlays.²⁵ It is characterized by frosted glass surfaces that blur the background while maintaining high transparency.²⁴

Technical Execution of the Glass Effect

The glassmorphism effect is primarily driven by the `backdrop-filter` CSS property, which applies graphical effects like blurring to the area *behind* an element.⁹

CSS

```
.glass-card {  
  background: rgba(255, 255, 255, 0.15);  
  backdrop-filter: blur(10px);  
  -webkit-backdrop-filter: blur(10px);  
  border: 1px solid rgba(255, 255, 255, 0.3);  
  border-radius: 16px;  
}
```

This combination of low-opacity background and blur creates a sense of hierarchy without completely obscuring the context.²³ Cloudmorphism is an evolution of this concept, where the "glass" layer is replaced with "volumetric clouds." This is achieved by using "Aurora

backgrounds"—multi-colored, blurred gradients that shift over time, creating an atmospheric, nebulous effect.²⁴

Jellymorphism and Liquid Dynamics

"Jellymorphism" represents the fluid extreme of glassmorphism. In this style, elements not only blur the background but also exhibit liquid behaviors, such as surface tension and refraction.¹⁸ Implementation often involves using SVG blob filters to create "wobbly" shapes that respond to cursor movement.¹⁸ Advanced versions use the "Liquid Glass" technique, where CSS @property is used to animate custom variables for border-radius and shadow, allowing a card to "melt" or "ripple" when interacted with.¹⁸

Multi-Sensory Immersion and Sensory Theory

The ultimate objective of tactile UI is to transcend the visual and engage the user's other senses, primarily touch and sound. Multi-sensory UX is a design philosophy that integrates haptics and audio to reinforce the visual "morphisms" being presented.³²

Haptic Feedback: The Sense of Touch

Haptic feedback uses vibrations to communicate information. In tactile UIs, haptics provide a physical confirmation of a virtual action, bridging the gap between the glass screen and the user's nervous system.³³ For example, a "squishy" claymorphic button might trigger a heavy, damp vibration to simulate the resistance of clay, whereas a glassmorphic element might trigger a crisp, high-frequency "ping".³³

- **Vibration API:** Standard web API for mobile haptics.
- **Tactile Waves:** A library for sound-to-touch sensory substitution, allowing audio waveforms to be converted into real-time tactile vibrations on body-worn hardware.³⁶
- **Assistive Touch:** Haptic technologies that simulate Braille or physical textures for visually impaired users, significantly improving accessibility.³⁴

Tactile Sound Design

Audio cues in tactile UIs are designed to mimic the "acoustic properties" of the material being simulated.³² A "fabricmorphic" UI might use soft, frictional sounds (swishes), while a "papermorphic" UI would use the sound of crisp folding or tearing.³⁷ Libraries like "UI: Tactile" provide high-fidelity recordings of real-world clicks and taps specifically for integration into digital products.³⁷

Performance Architecture and Implementation

Strategies

Tactile UIs are computationally expensive. The heavy use of backdrop-filter, complex shadows, and SVG noise generators can lead to "jank" and high GPU usage.²³ Developers must employ sophisticated strategies to maintain performance.

CSS Cascade Layers and Modern Styling

The introduction of CSS Cascade Layers (@layer) has improved the management of complex "morphic" styles. By separating foundational resets, theme layers (clay, glass, paper), and component-specific overrides, developers can control specificity without resorting to !important.⁴⁰ This is particularly useful in design systems like Prime, which manage over 3,000 components with integrated variables for color, spacing, and radius.⁸

Optimization Techniques for Morphic UIs

- 1. **Lazy Loading of Animations:** Using next/dynamic to load heavy animation libraries only when needed.¹⁰
- 2. **GPU Offloading:** Using transform: translate3d(0) to force hardware acceleration for blurred elements.¹⁰
- 3. **Static Texture Fallbacks:** For low-end devices, replacing real-time SVG turbulence with static bitmaps or CSS gradients to reduce filter overhead.²⁰
- 4. **Reduce Motion Media Query:** Respecting the user's OS preference to disable spring physics and complex blurs for users with vestibular sensitivities.²³

Implementation Challenge	Strategy	Impact
GPU Load (Blur)	Limit simultaneous filters ²⁹	Smoother scrolling on mobile ²³
Animation Weight	Use LazyMotion wrapper ¹⁰	Reduced initial bundle size ¹⁴
Shadow Complexity	Consolidate box-shadow ⁹	Faster paint times ⁴¹
Texture Scaling	Resolution-independent SVG filters ²⁰	Sharpness across all DPIs ¹⁹

Siliconmorphism and Gelmorphism: The Industrial Frontier

As the design industry moves toward 2026, "Siliconmorphism" and "Gelmorphism" represent the new frontier of industrial-aesthetic UIs. Siliconmorphism mimics the matte, semi-translucent appearance of high-grade silicone—a material that is soft to the touch but visually precise.²³ This requires a lower saturation in the blur and a specific "velvety" highlight achieved through `feComponentTransfer` to adjust the alpha channel of the shadows.²⁰

Gelmorphism, conversely, emphasizes the high-gloss, refractive nature of liquid gels. This is implemented through "Refractive Shaders" in Three.js, where the mesh of the UI element acts as a lens, distorting the "Aurora background" beneath it.¹⁸ These styles are increasingly favored in the cryptocurrency and metaverse sectors to emphasize the "futuristic" and "high-tech" nature of their platforms.²⁴

Conclusion: The Integrated Future of Surface Design

The evolution of digital surface design from skeuomorphism to the current "morphic" landscape reflects a deepening understanding of human perception and browser technology. Claymorphism provides an approachable, inclusive evolution of 3D design, while glassmorphism and cloudmorphism offer sophisticated atmospheric depth. The integration of SVG filters for fabric and paper textures, combined with the physics of spring-based animation, has created a new standard for tactile interactivity.

The transition toward multi-sensory immersion—integrating haptics and spatial audio—ensures that the digital experience is no longer confined to the visual plane. As developers master the performance trade-offs of these GPU-intensive styles, the "morphic" trend will continue to evolve into even more refined materials like silicon and gel, providing a digital world that is as rich, textured, and responsive as the physical one it mimics. The future of UI is not flat; it is tactile, voluminous, and profoundly sensory.

Works cited

1. All about claymorphism and how to create it on your own - Bamboo Lab, accessed March 15, 2026, <https://bamboolab.eu/blog/design/all-about-claymorphism-and-how-to-create-it-on-your-own>
2. Neumorphism vs. Skeuomorphism vs Glassmorphism – UI Design Trends Explained - Uinkits, accessed March 15, 2026, <https://www.uinkits.com/blog-post/neumorphism-vs-skeuomorphism-vs-glassmorphism---ui-design-trends-explained>

3. Glassmorphism vs. Claymorphism vs. Skeuomorphism: 2025 UI Design Guide | by Gennadiy Oleinyk | Bootcamp | Medium, accessed March 15, 2026,
<https://medium.com/design-bootcamp/glassmorphism-vs-claymorphism-vs-skeuomorphism-2025-ui-design-guide-e639ff73b389>
4. Skeuomorphism vs. flat design vs material design - 99Designs, accessed March 15, 2026,
<https://99designs.com/blog/trends/skeuomorphism-flat-design-material-design/>
5. Do you prefer sceuomorphism or flat design? - Quora, accessed March 15, 2026,
<https://www.quora.com/Do-you-prefer-sceuomorphism-or-flat-design>
6. Skeuomorphism vs Neumorphism vs Glassmorphism: Which UI Design Style Should You Choose?, accessed March 15, 2026,
<https://designworklife.com/skeuomorphism-vs-neumorphism-vs-glassmorphism-which-ui-design-style-should-you-choose/>
7. Claymorphism: Will It Stick Around? - Smashing Magazine, accessed March 15, 2026,
<https://www.smashingmagazine.com/2022/03/claymorphism-css-ui-design-trend/>
8. Claymorphism tutorial – How to mimic clay in UI design | UXMISFIT ..., accessed March 15, 2026,
<https://uxmisfit.com/2022/04/29/claymorphism-tutorial-how-to-mimic-clay-in-ui-design/>
9. How To Create Claymorphism Using CSS | SquarePlanet, accessed March 15, 2026,
<https://hype4.academy/articles/coding/how-to-create-claymorphism-using-css>
10. A Guide to Framer Motion React Animation | Magic UI, accessed March 15, 2026,
<https://magicui.design/blog/framer-motion-react>
11. Motion - UI Playbook, accessed March 15, 2026,
<https://uiplaybook.dev/play/motion>
12. React Spring Guide That Makes Framer Motion Look Heavy - JavaScript in Plain English, accessed March 15, 2026,
<https://javascript.plainenglish.io/react-spring-guide-that-makes-framer-motion-look-heavy-da24da606666>
13. How to Create Physics-Based Spring Animations with Custom Damping in JavaScript, accessed March 15, 2026,
<https://dev.to/hexshift/how-to-create-physics-based-spring-animations-with-custom-damping-in-javascript-1e08>
14. Comparing the best React animation libraries for 2026 - LogRocket Blog, accessed March 15, 2026,
<https://blog.logrocket.com/best-react-animation-libraries/>
15. Introduction - React Three Fiber, accessed March 15, 2026,
<https://r3f.docs.pmnd.rs/getting-started/introduction>
16. Examples - React Three Fiber, accessed March 15, 2026,
<https://r3f.docs.pmnd.rs/getting-started/examples>

17. How to create animated shaders with three.js - YouTube, accessed March 15, 2026, <https://www.youtube.com/watch?v=e2ntx-fyXaE>
18. Liquid Glass design using CSS - YouTube, accessed March 15, 2026, <https://www.youtube.com/watch?v=kTOLrsDIKQA>
19. Creating Patterns With SVG Filters - CSS-Tricks, accessed March 15, 2026, <https://css-tricks.com/creating-patterns-with-svg-filters/>
20. SVG Filter Effects: Conforming Text to Surface Texture with ..., accessed March 15, 2026, <https://tympanus.net/codrops/2019/02/12/svg-filter-effects-conforming-text-to-surface-texture-with-fedisplacementmap/>
21. Paper Quilling designs, themes, templates and downloadable graphic elements on Dribbble, accessed March 15, 2026, <https://dribbble.com/tags/paper-quilling>
22. Quilled Paper Art - Dribbble, accessed March 15, 2026, https://dribbble.com/tags/quilled_paper_art?page=2
23. Top CSS Glassmorphism Examples to Explore - DEV Community, accessed March 15, 2026, <https://dev.to/er-raj-aryan/top-css-glassmorphism-examples-to-explore-1l2c>
24. [2024] UI Design Trends - Pixso, accessed March 15, 2026, <https://pixso.net/news/ui-trends/>
25. 2021's NEW WEB DESIGN TREND - Intro, CSS How To & Longevity Discussion - YouTube, accessed March 15, 2026, <https://www.youtube.com/watch?v=BGtkqPrjkos>
26. Glassmorphism in 5 Easy Steps | UI Trends 2021 - YouTube, accessed March 15, 2026, https://www.youtube.com/watch?v=AdNOcgK_6Bs
27. Glassmorphism - UI Design Trend 2020 - YouTube, accessed March 15, 2026, <https://www.youtube.com/watch?v=GLHut0BM9nc>
28. Glassmorphism CSS Generator - Glass UI, accessed March 15, 2026, <https://ui.glass/generator/>
29. 12 Glassmorphism UI Features, Best Practices, and Examples, accessed March 15, 2026, <https://uxpilot.ai/blogs/glassmorphism-ui>
30. How to Create Glassmorphism Effect? - GeeksforGeeks, accessed March 15, 2026, <https://www.geeksforgeeks.org/css/how-to-create-glassmorphism-effect/>
31. Colorful Glowing Liquid Bowl Animation Effects | Html CSS Animation Effects - YouTube, accessed March 15, 2026, <https://www.youtube.com/watch?v=HCO0-IDCdDM>
32. Multi-Sensory UX: Combining Sight, Sound, and Touch in Digital Products | QodeQuay, accessed March 15, 2026, <https://www.qodequay.com/multi-sensory-ux-digital-products>
33. Multi-Sensory UX: Integrating Haptics, Sound, and Visual Cues to Enhance User Interaction, accessed March 15, 2026, <https://wings.design/insights/multi-sensory-ux-integrating-haptics-sound-and-visual-cues-to-enhance-user-interaction>
34. How Multi-Sensory Design is Transforming Web Accessibility, accessed March 15,

- 2026,
<https://rtctek.com/how-multi-sensory-design-is-transforming-web-accessibility/>
35. Multisensory Web Design: Exploring Haptic Feedback Trends, accessed March 15, 2026,
<https://hfbtechnologies.com/how-haptic-feedback-is-revolutionizing-online-experiences/>
36. Funkatronics/TactileWaves: An Android/Java library for real-time sound-to-touch sensory substitution - GitHub, accessed March 15, 2026,
<https://github.com/Funkatronics/TactileWaves>
37. UI: Tactile | UI Sound Effects Library | asoundeffect.com, accessed March 15, 2026,
<https://www.asoundeffect.com/sound-library/ui-tactile/>
38. UI: Tactile - Sound Armoury, accessed March 15, 2026,
<https://www.soundarmoury.co.uk/ui-tactile>
39. Tactile. Touch the sound to create cinematic ambients. | Sampleson, accessed March 15, 2026, <https://sampleson.com/tactile.html>
40. Cascade Layers Guide - CSS-Tricks, accessed March 15, 2026,
<https://css-tricks.com/css-cascade-layers/>
41. Applying a filter to an SVG image causes it to pixellate · Issue #1769 · fabricjs/fabric.js, accessed March 15, 2026,
<https://github.com/fabricjs/fabric.js/issues/1769>